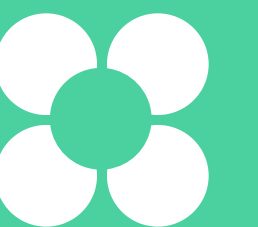


Многопоточное программирование. Работа с потоками



План занятия

- 1 Многопоточное программирование
- 2 Thread и Runnable
- 3 Future и пулы потоков

Многопоточное программирование

Филипп Воронов
Teamlead в поиске VK

Программа

Программа во время выполнения представляет один или несколько процессов, которым ОС выделила память для хранения данных и процессорное время

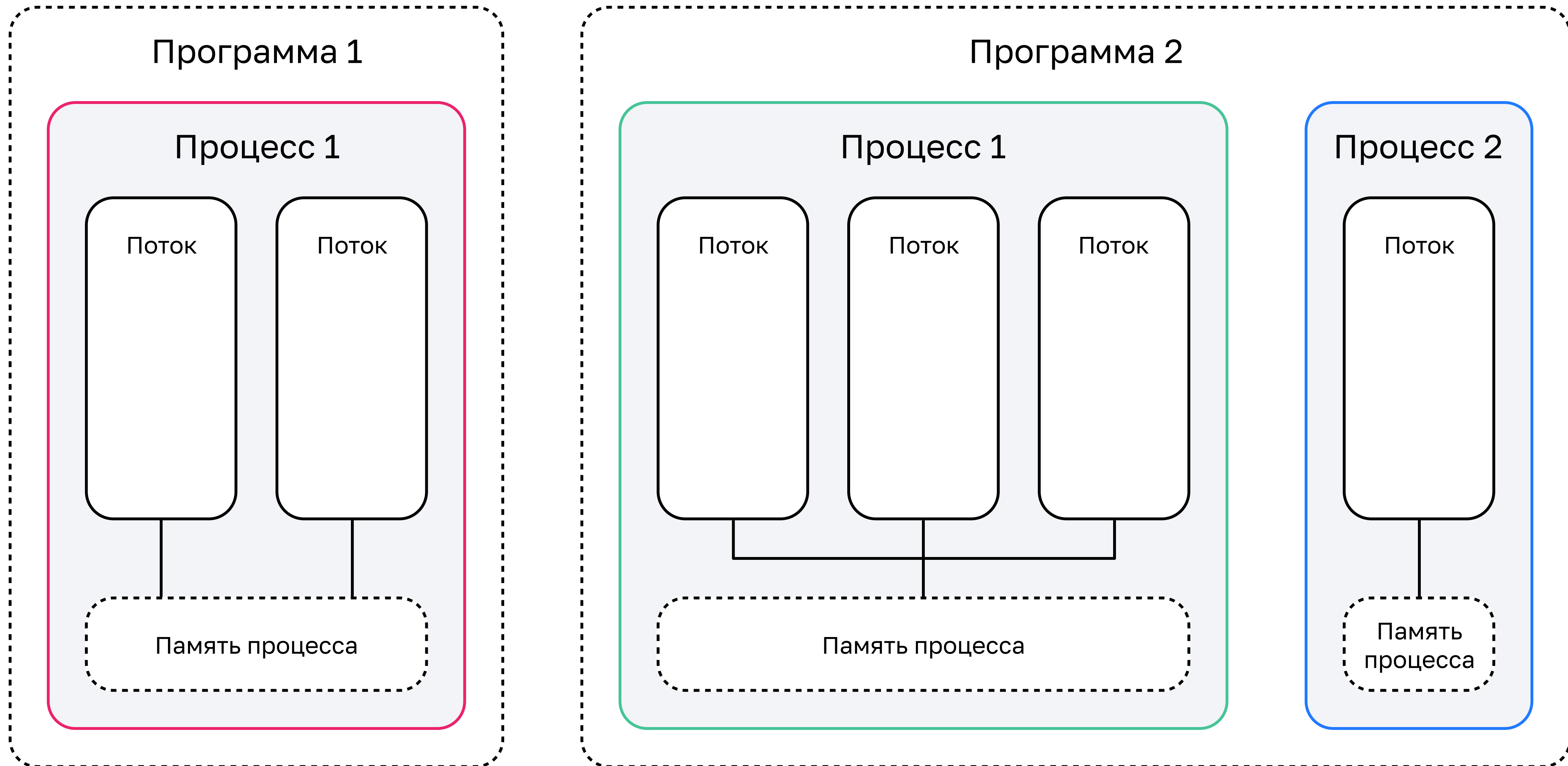
Процесс

Процесс — это совокупность
данных и кода, работающего
с этими данными

Поток

Поток — выполняемая
последовательность инструкций,
часть процесса

Многопоточное программирование



Многопоточное программирование

- ⓘ В самой доступной формулировке **закон Мура** говорит, что каждые **18** месяцев мощность процессоров на рынке **возрастает вдвое**

Но память не увеличивается так быстро

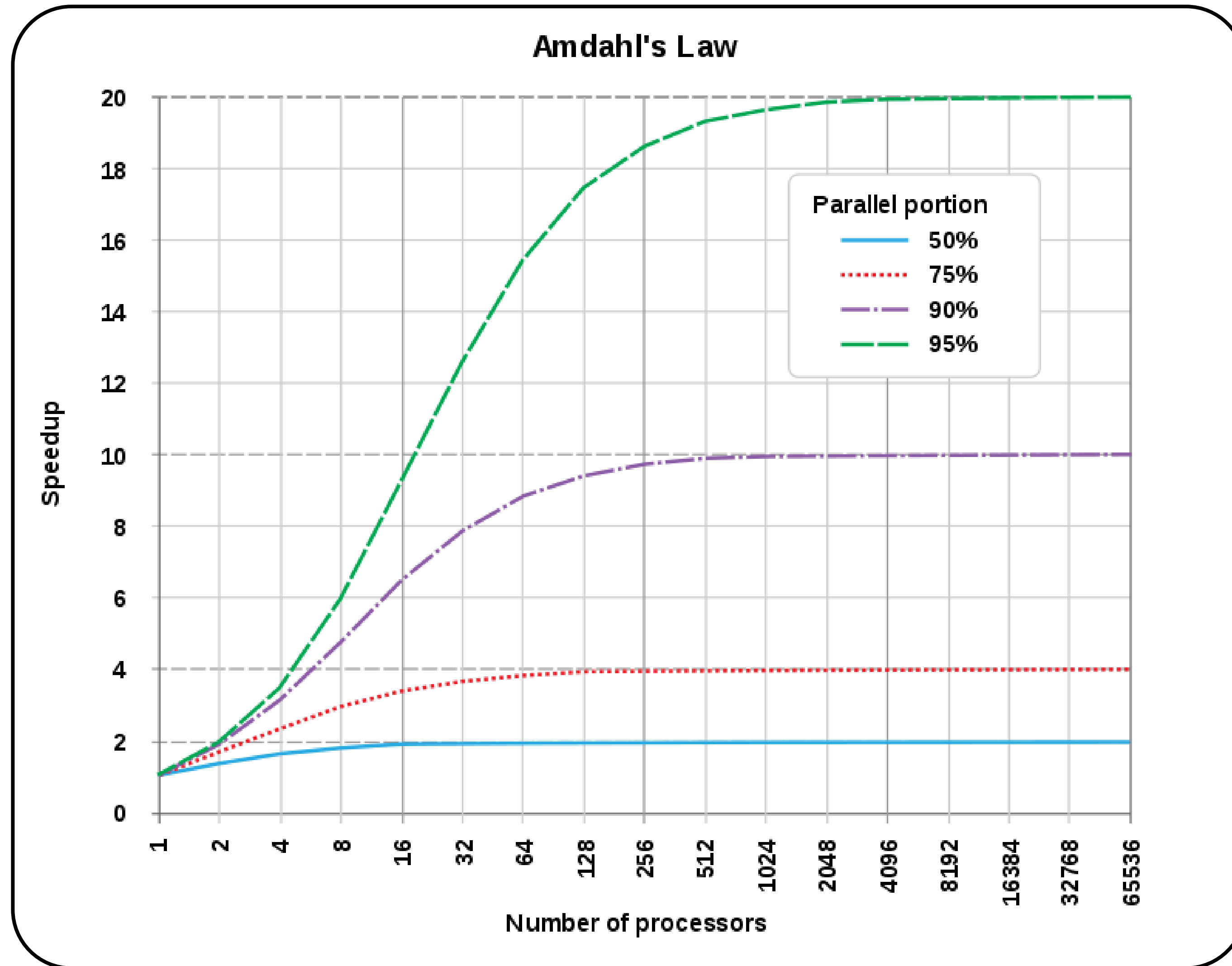


Многопоточное программирование

- ! **Закон Амдала** говорит, что в случае, когда задача разделяется на несколько частей, суммарное время её выполнения на параллельной системе не может быть меньше времени выполнения самого медленного фрагмента



Многопоточное программирование



Thread и Runnable

Филипп Воронов
Teamlead в поиске VK

Работа с потоками

❗ Минимальная единица многопоточности — это поток

Способы создания потока в Java:

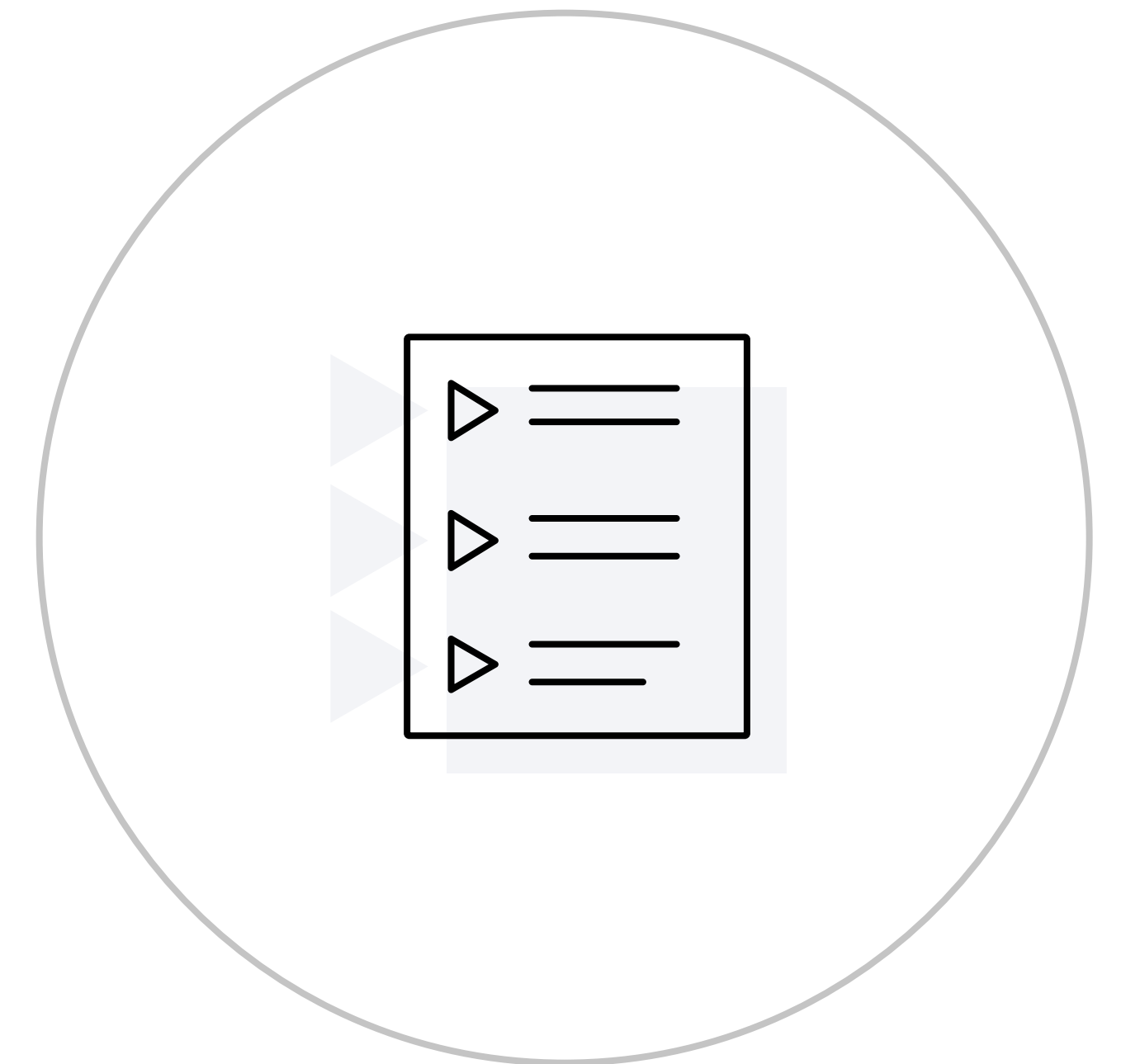
- 1 реализация наследника класса Thread (поток)
- 2 реализация интерфейса Runnable
- 3 реализация интерфейса Callable



Класс Thread

Основные методы класса Thread:

- `start()` — запуск потока
- `interrupt()` — остановка потока
- `getId()` — получение идентификатора процесса
- `setName()/getName()` — установка и получение имени потока
- `setPriority()/getPriority()` — установка и получение приоритета
- `getState()` — получение статуса потока
- `isInterrupted()` — вернёт `true`, если выполнение потока было остановлено
- `isAlive()` — вернёт `true`, если поток работает
- `isDaemon()` — вернёт `true`, если поток является потоком-демоном



Класс Thread

Метод `interrupt()` используется, чтобы останавливать потоки:

```
public class Main {  
  
    public static void main(String[] args) throws Exception {  
        MyThread myThread = new MyThread();  
        myThread.start();  
        myThread.interrupt();  
    }  
}
```

```
public class MyThread extends Thread {  
  
    @Override  
    public void run() {  
        while(true) {  
            if(isInterrupted()) return;  
            //TODO: код выполняемого процесса  
        }  
    }  
}
```

Класс Thread

Чтобы назначить и получить имя потока, используются методы `getName()` и `setName()`:

```
public class Main {  
    private static final Logger LOG = LoggerFactory.getLogger(Main.class);  
  
    public static void main(String[] args) throws Exception {  
        MyThread myThread = new MyThread();  
        myThread.setName("Поток-загрузчик 1");  
        LOG.debug("Запускаю поток с именем {} и идентификатором {}", myThread.getName(),  
            myThread.getId());  
        myThread.start();  
    }  
}
```

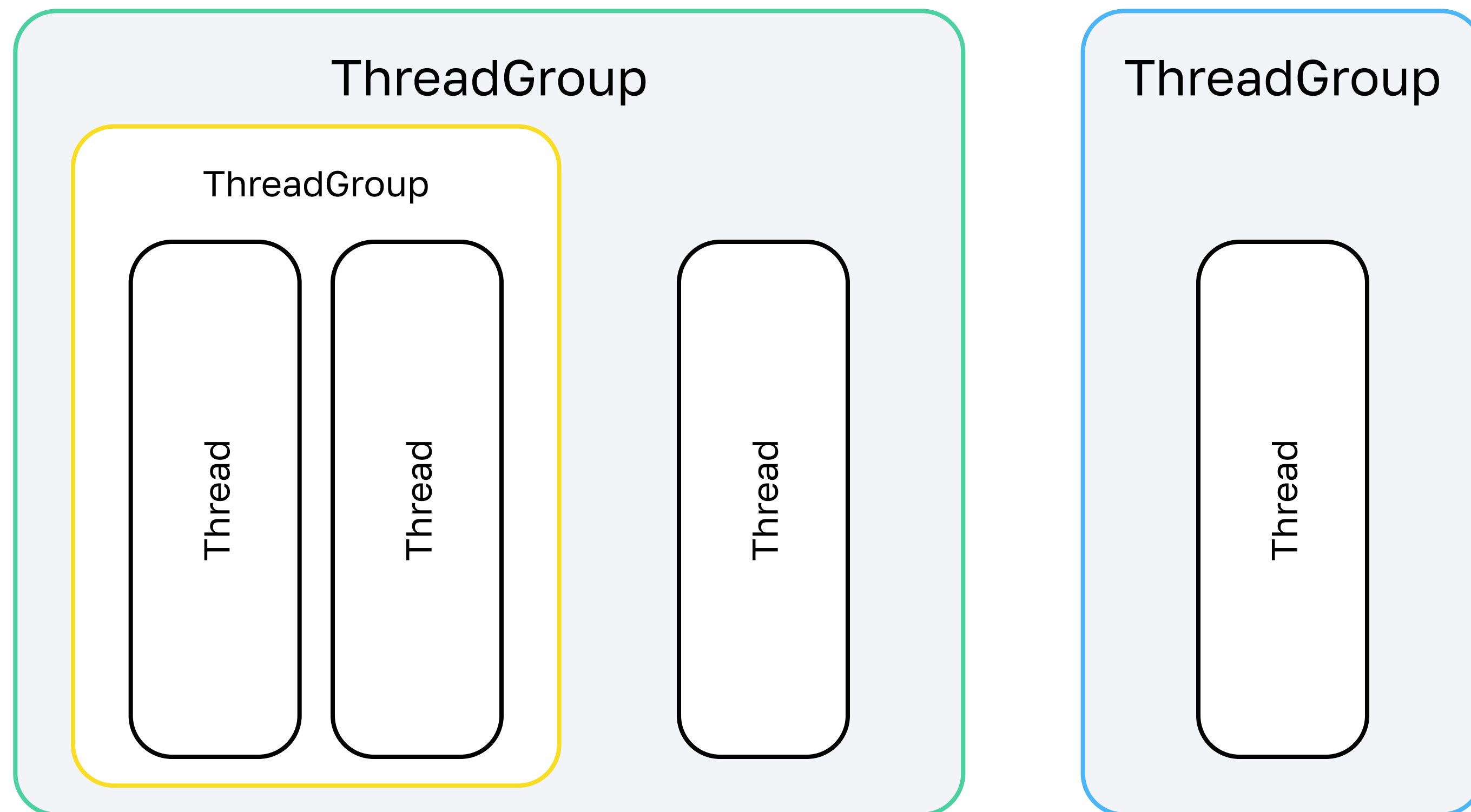

Класс Thread

Чтобы поток стал потоком-демоном, надо вызвать метод `setDaemon()`:

```
public class Main {  
  
    public static void main(String[] args) throws Exception {  
  
        MyThread myThread = new MyThread();  
  
        //Помечаем поток, как поток-демон  
        myThread.setDaemon(true);  
    }  
}
```

Группировка потоков – ThreadGroup

Для **логической группировки потоков** существует класс **ThreadGroup**, объекты которого могут содержать потоки и группы потоков, помогая строить иерархию потоков



Группировка потоков – ThreadGroup

```
// Главная группа потоков
ThreadGroup mainGroup = new ThreadGroup("main group");

// Вторая группа потоков, входящая в главную
ThreadGroup group1 = new ThreadGroup(mainGroup, "group-1");

// Добавление потоков в группы
final Thread thread1 = new Thread(group1, myRunnable);
final Thread thread2 = new Thread(group1, myRunnable);
final Thread thread3 = new Thread(mainGroup, myRunnable);

// Запуск потоков
thread1.start();
thread2.start();
thread3.start();

// Понижаем приоритет у всех потоков группы group-1
group1.setMaxPriority(4);

// Завершаем все потоки одним вызовом
mainGroup.interrupt();
```


Future и пулы потоков

Филипп Воронов
Teamlead в поиске VK

Работа с потоками

Чтобы запустить задачу в отдельном потоке и получить результат, в Java предусмотрен интерфейс `java.util.concurrent.Future`, который имеет методы:

- `isCancelled()` — вернёт `true`, если задача была отменена
- `isDone()` — вернёт `true`, если задача была завершена
- `get()` — получить результат после завершения задачи
- `cancel()` — отмена задачи

Работа с потоками

- ❗ Отменённую задачу запустить нельзя. Если задача во время отмены уже выполнялась, то она завершится сразу, если был вызван `cancel()` с параметром `mayInterrupt == true`
- ❗ Завершённой задача считается не только в случае её правильного завершения с результатом, но и в случае выброса исключения во время выполнения

Работа с потоками

Одна из самых популярных реализаций интерфейса **Future** — класс **FutureTask**:

```
public class Main {  
    public static void main(String[] args) throws ExecutionException, InterruptedException {  
        final MyCallable myCallable = new MyCallable();  
        final FutureTask<String> stringFutureTask = new FutureTask<>(myCallable);  
        new Thread(stringFutureTask).start();  
  
        final String resultOfTask = stringFutureTask.get();  
        System.out.println("Результат выполнения задачи: " + resultOfTask);  
    }  
}
```

❗ Вызов методов **run()** или **call()** не приводит к выполнению задач в отдельном потоке

Executor

В Java определён стандартный интерфейс, описывающий метод запуска Runnable-объекта без создания специального потока — `java.util.concurrent.Executor`:

```
public interface Executor {  
  
    void execute(Runnable command);  
  
}
```


ExecutorService

ExecutorService предлагает больше методов для запуска и управления выполнением задач. Он позволяет запускать **Runnable**- и **Callable**-объекты без лишних обёртываний в **Thread** и **FutureTask**. Достаточно использовать метод **submit()**:

```
// запуск Callable-объекта
// сразу получаем объект Future для управления задачей и получения результата
<T> Future<T> submit(Callable<T> task);

// запуск Runnable-объекта
// сразу получаем объект Future для управления задачей
Future<?> submit(Runnable task);

// запуск Runnable-объекта (например FutureTask<T>)
// сразу получаем объект Future для управления задачей и получения результата
<T> Future<T> submit(Runnable task, T result);
```

ExecutorService

ExecutorService позволяет определять методы для запуска нескольких задач сразу или запуска всех задач ради решения хотя бы одной:

```
// запуск нескольких Callable-объектов
<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)
    throws InterruptedException;

// запуск нескольких Callable-объектов
// задачи, решение которых заняло больше времени, отменяются автоматически
<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks,
    long timeout, TimeUnit unit)
    throws InterruptedException;

// запуск нескольких Callable-объектов
// возврат результата первой успешно выполненной задачи. Остальные отменяются
<T> T invokeAny(Collection<? extends Callable<T>> tasks)
    throws InterruptedException, ExecutionException;

// запуск нескольких Callable-объектов
// задачи, решение которых заняло больше времени, отменяются автоматически
// возврат результата первой успешно выполненной задачи. Остальные отменяются
<T> T invokeAny(Collection<? extends Callable<T>> tasks,
    long timeout, TimeUnit unit)
    throws InterruptedException, ExecutionException, TimeoutException;
```

ExecutorService

Если нам больше не нужен **ExecutorService**, мы должны правильно его выключить. Для этого есть несколько методов

- **shutdown()** — перестаёт запускать задачи, отменяет все запущенные, соблюдая порядок
- **shutdownNow()** — отменяет все запущенные задачи, возвращает их список
- **awaitTermination()** — ожидает завершения всех задач или таймаута и выключает сервис

ExecutorService

Существуют и методы, которые определяют состояние **ExecutorService**:

- **isShutdown()** — вернёт true, если сервис выключен одним из описанных выше методов
- **isTerminated()** — вернёт true, если сервис закончил выполнение всех отправленных задач

Пул потоков

Чтобы создать пул потоков, используется класс **Executors** и его статический метод **newFixedThreadPool**:

```
public class Main {  
    public static void main(String[] args) {  
  
        // Создаём пул потоков фиксированного размера  
        final ExecutorService threadPool = Executors.newFixedThreadPool(4);  
    }  
}
```


Пул потоков

Запуск задачи с помощью пула:

```
public class Main {  
    public static void main(String[] args) throws ExecutionException, InterruptedException {  
        // Создаём задачу с результатом типа String  
        Callable<String> myCallable = new MyCallable<>();  
        // Создаём пул потоков фиксированного размера  
        // В этом случае у нас будет 4 потока  
        final ExecutorService threadPool = Executors.newFixedThreadPool(4);  
        // Отправляем задачу на выполнение в пул потоков  
        final Future<String> task = threadPool.submit(myCallable);  
        // Получаем результат  
        final String resultOfTask = task.get();  
        // Завершаем работу пула потоков  
        threadPool.shutdown();  
    }  
}
```